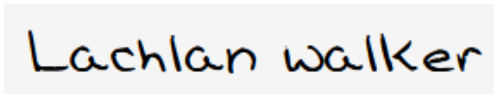
BE Final Report

Solving Optimisation Problems Using Classical and Quantum-Inspired Methods

Declaration of Authorship

I have read and understood the university's policy on plagiarism. I declare that all material in this assessment is my own work except where there is clear acknowledgement and appropriate reference to the work of others.

Signed: 

Date: 19/04/2026

Contents

Acronyms

QUBO Quadratic unconstrained binary optimisation

SDK Software Development Kit

SA Simulated Annealing

QA Quantum Annealing

SQA Simulated Quantum Annealing

ME Minor Embedding

BQM Binary Quadratic Model

MIQP Mixed Integer Quadratic Programming

PO Portfolio Optimisation

Chapter 1

Introduction

1.1 Background

Combinatorial optimisation problems are some of the most common computational problems facing plenty of fields such as logistics, finance, telecommunications, scheduling and machine learning. These problems involve selecting the best decision from a large set of discrete possibilities, and the number of feasible solutions often grows rapidly with problem size. Because of this, combinatorial optimisation continues to attract research interest, since many NP-hard problems in this class admit formulations as Ising or QUBO models[1].

A central difficulty is that no single method performs best on every problem class. Exact methods give guarantees on solution quality but may scale poorly with problem size, while heuristic and metaheuristic methods are more flexible and may scale better, but typically sacrifice optimality guarantees for speed. Benchmarking studies are therefore important in evaluating methods under controlled conditions rather than relying on theoretical expectations alone.

One of the best-known heuristics is simulated annealing[2] based on the physical cooling of metals. It uses a probabilistic search process in which worse solutions may be accepted early, allowing the method to escape local minima. Its appeal lies in its simplicity, modest implementation, and broad applicability. Quantum methods have also attracted a growing interest. Quantum annealing employs quantum fluctuations, similar to thermal fluctuations in simulated annealing[3], but physical quantum annealers are constrained by limited

connectivity and device noise. Simulated quantum annealing instead explores similar search behaviour on classical hardware.

This study also includes two exact methods, Google OR-Tools CP-SAT as an open-source classical solver for discrete optimisation, and Gurobi as a high-performance commercial solver. In this work, Gurobi is used primarily to provide high-quality reference solutions and bounds against which the solution quality of the heuristic methods can be assessed.

Max-Cut is used as a benchmark problem because it can be easily written in ways compatible with all four solvers and has a long history as a benchmark for optimisation algorithms[4]. Benchmark instances are based on Erdős-Rényi graphs across multiple sizes and densities, to systematically investigate scale and structural complexity.

The study compares the following four solvers: simulated annealing (SA); simulated quantum annealing (SQA); Gurobi Mixed Integer Quadratic Programming (MIQP); and Google's operations research OR-Tools Constraint Programming SAT Solver (CP-SAT).

1.2 Motivation

The motivation for this project is to compare optimisation methods in a practical and balanced way. Increasing the problem size and complexity makes exact approaches difficult to use within realistic runtime and memory limits, so it is important to examine which methods best balance computational cost with solution quality.

A second motivation is the need to assess quantum-inspired methods against strong classical baselines. Max-Cut is widely used in annealing-based optimisation studies because of its natural Ising-type formulation, but results are only meaningful when interpreted alongside established classical methods. The question often is not whether one method is always superior, but under what conditions it is useful. This comparison is especially relevant under

limited computational resources. The project was implemented in Google Colab, which places limits on runtime, memory and session length. Using Erdős-Rényi graphs across different sizes and expected degrees also allows the benchmark to examine how solver behaviour changes with both scale and graph density.

1.3 Project Aims

The aim of this project is to benchmark classical and quantum-inspired methods for solving the Max-Cut problem under controlled conditions. Simulated Annealing, Simulated Quantum Annealing, Gurobi MIQP, and Google OR-Tools CP-SAT are evaluated on Erdős-Rényi graph instances with varying size and density. The study compares solution quality, runtime, scalability, and sensitivity to graph structure, while assessing the practical suitability of each method under computational constraints. The goal is to provide an evidence-based comparison of solver performance across different problem regimes. This project is intended as a practical benchmarking study, not as an attempt to claim any broad quantum advantage.

1.4 Scope of the Project

The scope of this project is limited to a practical benchmarking study of the four optimisation methods mentioned above, applied to Max-Cut instances on unweighted Erdős-Rényi graphs. The study focuses on empirical comparison of objective quality, runtime, scalability, density effects, and practical suitability under Google Colab limits. It does not try to survey all optimisation methods, develop new algorithmic theory, show approximation results, or assert any general quantum advantage. The conclusions, therefore, are restricted to the selected methods, benchmark instances, parameter choices, and compute environment used in this project.

1.5 Report Structure

The remainder of the report is arranged as follows. Chapter 2 reviews relevant background literature and identifies the research gap filled by the project. Chapter 3 defines the benchmark problem, dataset, and evaluation measures. Chapter 4 describes the study methodology, solver implementations and experiment design. In Chapter 5 the benchmarking results for selected instance regimes are presented and discussed. Finally, Chapter 6 discusses possible future work and ethical and sustainability issues relevant to the project.

Chapter 2

Literature Review

2.1 The Max-Cut Problem

The Max-Cut problem is defined for an undirected graph $G(V, E)$ and seeks to partition the set of vertices into two disjoint subsets such that the number, or total weight, of edges crossing between the two subsets is maximised. For a weighted graph with edge weights w_{ij}, the objective is:

$$maximise \sum_{(i,j)\in E} w_{ij}(x_i + x_j - 2x_i x_j), \qquad x_i \in \{0,1\}$$

Where x_i indicates which side of the partition vertex i belongs to, and w_{ij} is the weight of edge (i, j). For the unweighted case, $w_{ij} = 1$ for all edges. Max-Cut is a standard NP-hard combinatorial optimisation problem[5] and has been used in the evaluation of exact and heuristic methods[6].

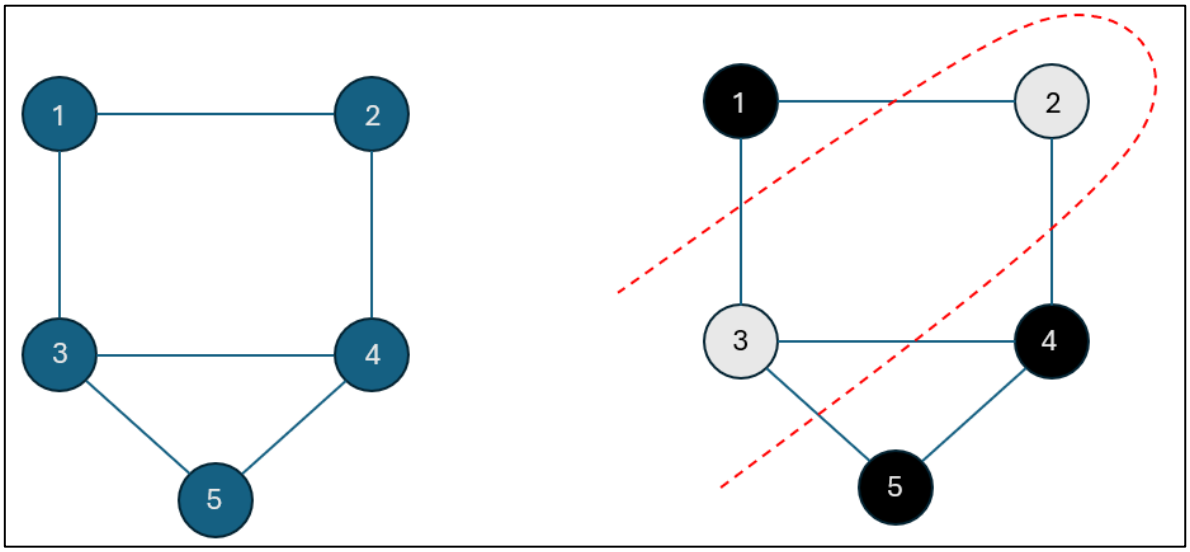


Fig 2.1: Example of an optimal Max-Cut partition on an undirected graph. The highlighted cut shows the 5 edges that contribute to the objective value.

Max-Cut was used as a benchmark problem as it is well defined[6], widely recognised in the optimisation literature, and simple to generate instances across a range of node sizes and graph densities. This makes it practical for controlled comparisons of objective quality, runtime, and scalability across different solvers. The problem has also been used as a test case for algorithmic performance, including approximation methods based on semidefinite programming[4].

Max-Cut is also relevant because it allows for a direct binary quadratic or Ising-style representation[1], which are compatible with classical annealing-based and quantum or quantum-inspired methods. This is useful here because all four methods can be compared on a common benchmark without requiring a different problem domain for each solver, so differences in performance can be interpreted more directly in terms of the methods themselves, the instance size, and the effect of graph density.

2.2 Classical Techniques for Optimisation

Classical approaches to combinatorial optimisation problems may be broadly divided into heuristic and exact methods. These approaches differ mostly in their treatment of optimality and computational costs. Heuristic methods aim to produce good solutions in reasonable time, whereas exact methods attempt to guarantee optimality at the expense of scalability.

Heuristic methods are employed when problem instances become too large for exact approaches to be practical. One of the most established techniques in this category is simulated annealing (SA), which is based on the physical process of annealing in metals. The method performs a stochastic search by iteratively changing candidate solutions and accepting changes based on a probability that depends on the change in objective value and some temperature parameter. For any given change in the objective value $\Delta \mathrm{E} = E_{new} - E_{old}$ the probability of acceptance is:

$$P(\Delta E) = \begin{cases} 1, & \Delta E < 0 \\ exp\left(-\frac{\Delta E}{T}\right), & \Delta E > 0 \end{cases}$$

where T is a temperature parameter that is gradually reduced according to a cooling schedule. This allows the method to explore the solution space broadly at the initial stages and converge towards lower-energy configurations over time[2].

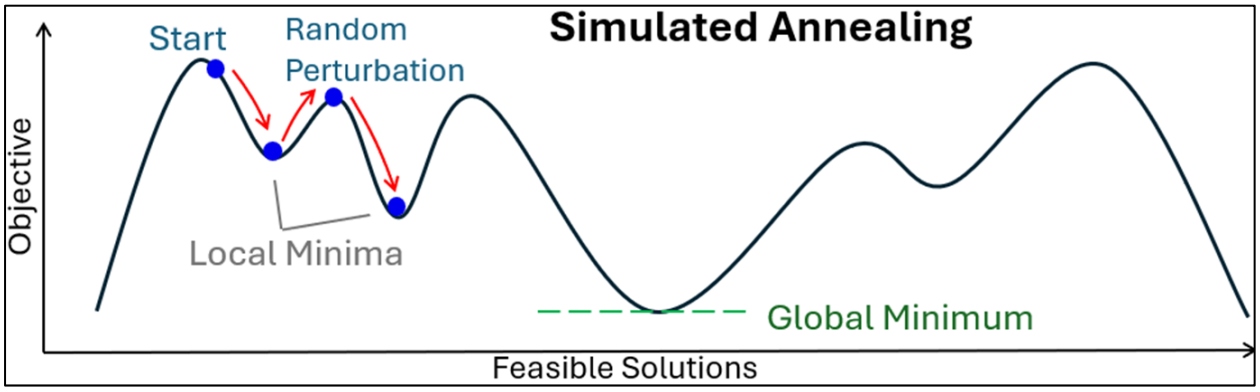


Fig 2.2: Conceptual energy landscape for Simulated Annealing, showing how SA escapes local minima by thermal perturbations

The effectiveness of SA depends heavily on the parameters selected, like the cooling schedule, number of iterations and neighbourhood structure[7]. Poor parameter selection can lead to premature convergence or inefficient exploration of the solution space. Several studies describe systematic ways of parameter tuning and hybridization with various other methods to enhance performance, including adaptive cooling schedules and hybrid metaheuristics that combine SA with local search or tabu search strategies[8]. Despite these improvements, the performance of SA remains highly problem-dependent, especially for large-scale or highly structured instances.

Beyond simulated annealing, several metaheuristic approaches have been developed for combinatorial optimisation problems, including genetic algorithms, tabu search, and hybrid methods. While such methods can achieve strong performance on specific problem classes, it is well established that no single heuristic consistently outperforms all others across all problem domains[9]. This motivates the use of controlled benchmarking studies, where

multiple algorithms are evaluated on the same set of instances to obtain meaningful comparisons[10].

However, exact optimisation techniques seek provably optimal solutions using mathematical programming formulations such as mixed-integer linear and quadratic programming. For the Max-Cut problem, the quadratic nature of the objective makes it well suited to formulation as a MIQP or QUBO. Branch and bound, cutting planes, and presolving are implemented in modern solvers to reduce the search space, but they suffer from fundamental scalability problems as computational effort increases with instance size[11]. In practice, solvers are frequently run with time limits, making them most useful as reference methods for benchmarking on small or medium-sized instances.

Recently, hybrid approaches combining constraint programming and satisfiability solving have gained prominence. A modern example of this class is the CP-SAT solver within Google OR-Tools[12]. It integrates constraint propagation, conflict-driven clause learning[13], and integer optimisation within a unified framework, operating directly on discrete variables and logical constraints rather than continuous relaxations. This can provide advantages for problems with strong combinatorial structure, but performance is highly formulation dependent. For problems such as Max-Cut, which are naturally expressed in quadratic form, the required reformulation into a constraint-based representation may add complexity.

A key factor for fair comparison is consistency of problem representation. In this project, all methods are applied to a common formulation of the Max-Cut problem to isolate differences in solver behaviour and avoid bias introduced by problem encoding[6]. Practical constraints such as runtime limits and memory usage are also important, especially in environments like Google Colab, where large-scale models may use up available memory and execution may exceed time limits.

2.3 Quantum Annealing and Quantum-Inspired methods

Quantum annealing (QA) is an optimisation technique based on quantum mechanics that seeks low-energy configurations of a system to encode a combinatorial optimisation problem. In contrast to simulated annealing, which relies on thermal fluctuations to explore the solution space, QA introduces quantum fluctuations that allow the system to change states via tunnelling effects. This should help avoid local minima more easily in problems with high energy barriers. Where classical methods must climb over such barriers, quantum tunnelling allows the system to pass through them, which can be useful in highly rugged energy landscapes[3][14].

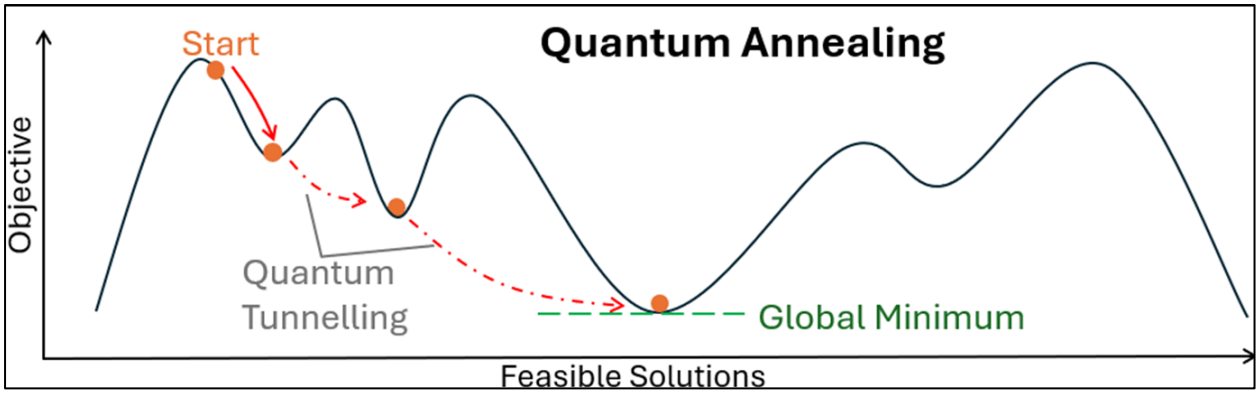


Fig 2.3: Conceptual energy landscape for Simulated Quantum Annealing, showing how SQA escapes local minima by tunnelling through barriers.

An optimisation problem is encoded into a time-dependent Hamiltonian of the form:

$$H(t) = A(t)H_D + B(t)H_P$$

Where H_D is the driver Hamiltonian causing quantum fluctuations and H_P is the problem Hamiltonian whose ground state encodes the optimal solution. The coefficients $A(t)$ and B(t) satisfy $A(0) \gg B(0)$ and $A(T) \ll B(T)$, so that the system is initialised in the ground state of H_D and gradually evolves towards H_P. Under ideal adiabatic conditions, the system remains in its ground state throughout this evolution, yielding an optimal or near optimal solution[15].

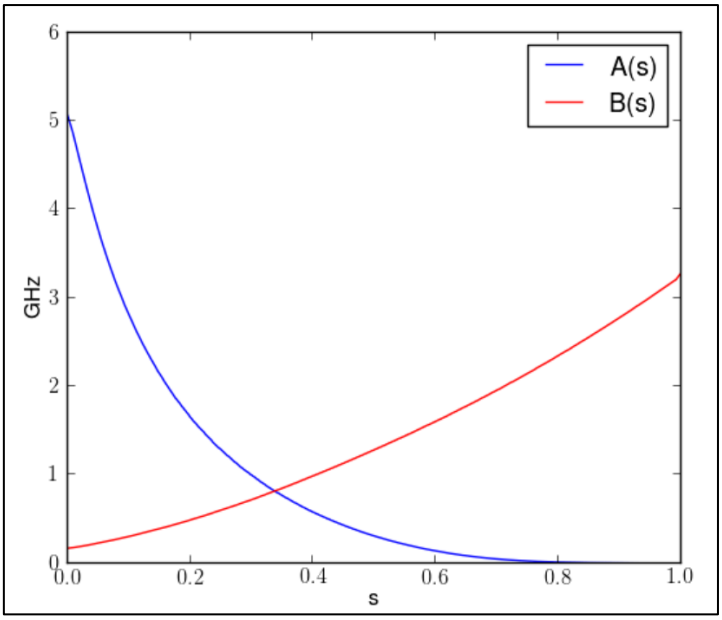


Fig 2.4: Diagram of an annealing schedule showing $A(t)$ and $B(t)$ crossing over time (source: [16])

In practice, factors such as finite annealing time, noise and hardware limitations cause deviations from ideal adiabatic behaviour. Physical implementations of QA have been developed by D-Wave Systems, whose devices solve problems expressed in Ising or QUBO form. The Quadratic Unconstrained Binary Optimisation (QUBO) model is a general framework for representing combinatorial optimisation problems as a quadratic objective over binary variables. A QUBO problem may be written as

$$\min_{x \in \{0,1\}^n} x^T Q x$$

where x is a vector of binary decision variables and Q is a real-valued $n \times n$ matrix encoding the relationships between variables. One reason QUBO is widely used is that many combinatorial optimisation problems can be transformed into this standard form, making it suitable for both classical heuristics and annealing-based methods. When a problem contains constraints, these are typically incorporated into the objective as quadratic penalty terms. For example, a constraint $g(x) = 0$ may be enforced by adding a term $\lambda g(x)^2$, where $\lambda > 0$ is chosen so that infeasible solutions become unfavourable.

These systems embed the problem onto a physical qubit topology. The most common D-Wave architecture is the Pegasus graph, which gives each qubit up to 15 connections to other qubits. However, even current quantum annealers suffer from practical limitations such as restricted qubit connectivity, limited coefficient precision and noise susceptibility[17]. If a problem graph is denser than the hardware topology allows, minor embedding is required to represent logical variables with chains of physical qubits, introducing additional overhead and reducing the effective problem size[18].

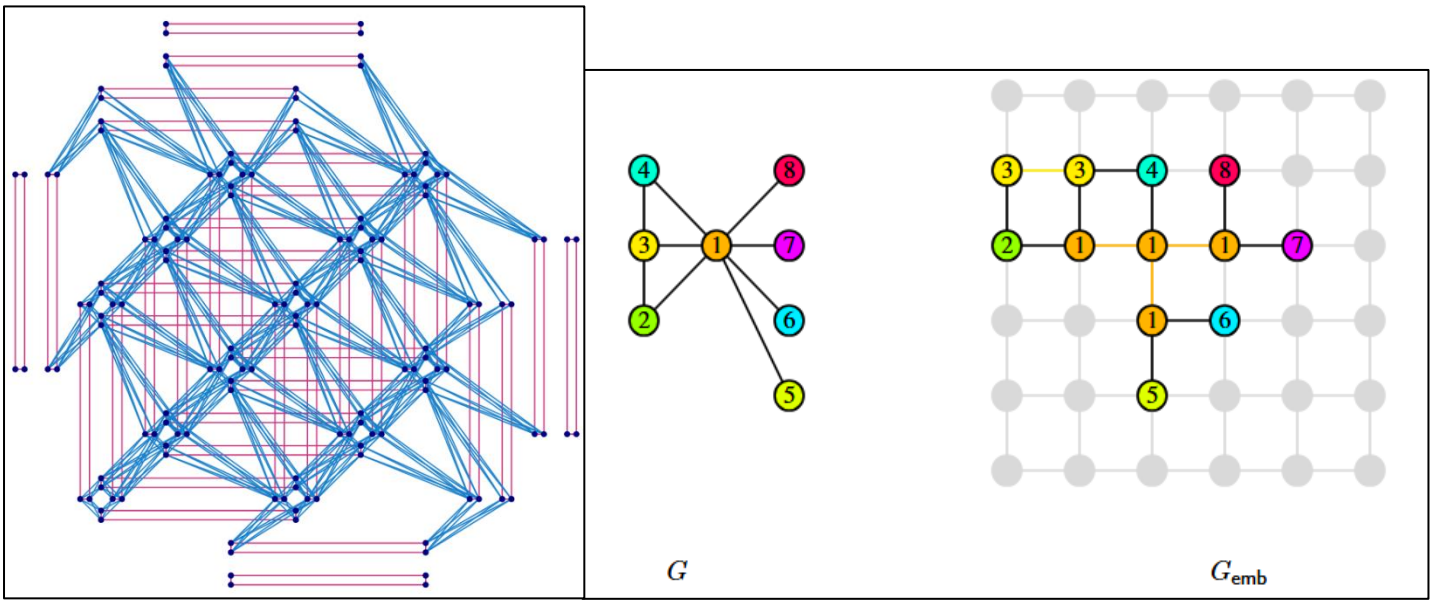


Fig 2.5: (Left) D-Wave Pegasus topology diagram [17]. (Right) Minor embedding of a sample problem G [18].

Due to these limitations, simulated quantum annealing (SQA) has been developed to approximate quantum annealing behaviour using classical computation. In order to apply either QA or SQA to a combinatorial optimisation problem, it must first be expressed in Ising form[1]. The QUBO objective can be mapped to an Ising Hamiltonian through the substitution:

$$x_i = \frac{1 + z_i}{2}, \qquad z_i \in \{-1, 1\}$$

Giving:

$$H_P = \sum_i h_i z_i + \sum_{i<j} J_{ij} z_i z_j$$

where z_i are spin variables, $h_i$corresponds to the diagonal entries of the QUBO matrix Q, and J_{ij} corresponds to the off-diagonal entries. This Ising Hamiltonian is then used as the problem Hamiltonian H_P in the annealing schedule described above.

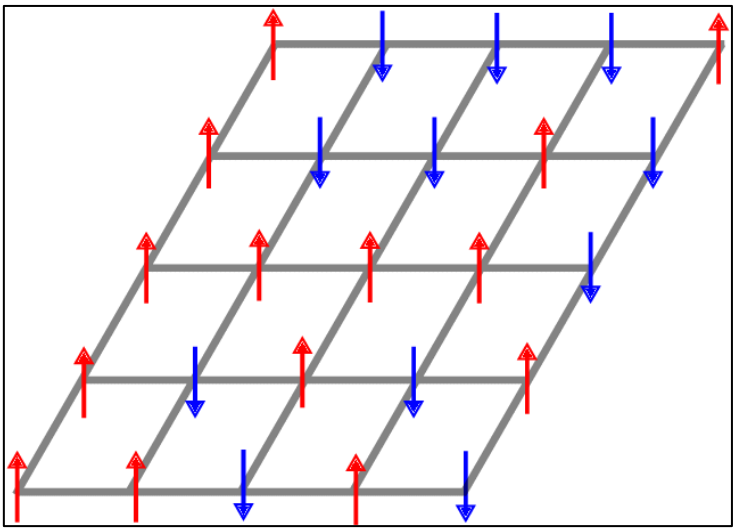


Fig 2.6: Example Ising model spin configuration on a 2-D lattice [19]

SQA uses this Ising representation to simulate tunnelling-like behaviour on classical hardware, exploring the solution space in a way different from purely thermal methods. Unlike physical quantum annealers, SQA does not suffer from the same hardware constraints as limited qubit count or noise, though problem size limitations imposed by the hardware topology still apply. Problem decomposition strategies can be used to extend SQA to larger instances by partitioning problems into subproblems that fit within the hardware constraints[20]. Any performance advantages observed remain algorithmic rather than quantum in nature and do not directly reflect the behaviour of physical quantum systems[21]. Benchmarking studies comparing QA, SQA, and classical methods have produced mixed results, with performance differences sensitive to problem structure, instance size, and parameter tuning[22][23][6].

In this project, SQA is treated as a quantum-inspired classical method rather than as evidence of quantum computational advantage. It is included alongside simulated annealing and exact methods to provide insight into how techniques derived from quantum mechanics perform compared to classical approaches when applied to a common benchmark problem. By maintaining consistent problem formulations across methods, the project aims to assess practical performance differences in terms of solution quality, runtime, and scalability.

2.4 Benchmarking Optimisation Methods

Benchmarking provides a structured comparison of optimisation algorithms under consistent conditions. For combinatorial optimisation problems, this is particularly important because there are many available algorithms and performance is highly dependent on problem structure, instance size, and parameter configuration[24].

A major decision-making point in benchmarking is the choice of performance metrics. In this project, the main metrics of interest are objective quality, runtime, and scalability. Objective quality measures how close a solution is to the optimal or best-known value, runtime measures the computational efficiency, and scalability refers to how solver performance changes as problem size increases. King et al. [25] introduced the "time-to-target" metric specifically for quantum annealing benchmarking, arguing that traditional ground-state success rate metrics become computationally prohibitive at scale and are distorted by hardware noise, making runtime-relative metrics more practical for fair comparison.

The structure of problem instances also influences solver performance. For graph-based problems like Max-Cut, graph size, density, and connectivity may affect the difficulty of the optimisation task [10]. Pelofske et al. [26] demonstrated this directly when benchmarking three generations of D-Wave annealers on Max-Cut and Max-Clique, finding that optimal chain strength during minor embedding varied with graph density, and that performance

differed between sparse and dense instances. A controlled dataset with systematically different parameters is thus preferable for methodical consistency of comparison.

Consistency of problem formulation is equally important. Differences with encoding can significantly impact solver performance, especially when comparing fundamentally different approaches such as heuristic search and constraint-based methods. Ensuring all solvers are applied to equivalent formulations reduces the risk that observed differences reflect encoding choices rather than algorithmic performance [27].

Solver configuration also plays a critical role. Many optimisation methods, particularly heuristics, are sensitive to parameter choices such as iteration limits or cooling schedules[7]. Inconsistent or poorly chosen parameters can lead to misleading comparisons, so benchmarking studies must either standardise settings or clearly document how parameters are selected. This extends to quantum methods as well, McGeoch [28] emphasises that when benchmarking quantum computers, performance claims are meaningless without reporting full runtime overheads, and that parameter choices such as annealing time can substantially change apparent performance.

Reproducibility is another consideration. Experiments should use fixed random seeds, consistent hardware environments, and clearly defined stopping criteria[24]. This is particularly important for stochastic methods such as SA and SQA, where run-to-run variability can affect both runtime and solution quality. Reporting results averaged across multiple runs is a standard practice to mitigate this[29].

Practical constraints shape the design of benchmarking as well. In this project, experiments are conducted within a cloud-based environment with limited memory and execution time, which affects the scale of instances that can be tested. Studies such as that by Quinton et al. [30]have highlighted that quantum annealers face similar practical constraints, noting that

computational time for hybrid solvers grows significantly as the number of variables increases even when solution quality remains competitive with classical approaches.

Overall, a well-designed benchmarking framework allows fair comparison of methods, with results reflecting genuine algorithmic trade-offs rather than artefacts of experimental design[31].

2.5 Research Gap to be Addressed

Existing literature provides extensive analysis of both classical and quantum-inspired optimisation methods but meaningful comparison between these approaches remains limited. The reported performance is sensitive to problem formulation, solver parameterisation, and hardware assumptions, so consistent conclusions between studies are difficult to achieve[24][28].

Many benchmarking studies use small or specially structured problem instances adapted to the strengths of different solvers[26][30]. Therefore, there is limited understanding of how these methods perform on larger-scale instances under real computational constraints, where memory usage and runtime dominate[30].

Quantum-inspired methods like SQA are frequently evaluated without considering important practical challenges like hardware connectivity constraints or embedding overhead[27]. The observed performance advantages may therefore not translate to real-world quantum implementations, and their relative benefits over classical approaches remain unclear[21].

A further limitation is the lack of studies integrating exact or hybrid solvers with heuristic methods in a consistent experimental framework despite their importance as quality baselines[6].

This project addresses these gaps with controlled benchmarking across multiple solvers on a common set of Max-Cut instances, focusing on solution quality, runtime, scalability, and sensitivity to graph structure under consistent formulations and constrained computational resources.

Chapter 3

Dataset Generation and Evaluation Metrics

3.1 Dataset Generation

For this project, we will only consider unweighted instances as this makes the dataset easier to generate. The focus of this section is on the construction of a dataset that enables the controlled evaluation of the solvers.

The dataset consists of Erdős-Rényi random graphs of the form $G(n, p)$ generated across a range of node counts (n) and expected degrees (d). Expected degree refers to the average number of neighbours each node has; larger values represent more interconnected graphs. Rather than specifying the edge probability or density (p) directly for all instances, graphs were parameterised using the expected degree, with the relationship $p = d/(n-1)$. This allows graph density to scale with the size of the instance, rather than specifying a set density which may be applicable for smaller instances but not for larger instances. This choice allows us to examine a wider range of interesting graph regimes.

Node counts are selected over several orders of magnitude, ranging from 100 up to $1{,}000{,}000$, while expected degree values span from sparse to moderately dense regimes. For each combination of (n, d), multiple independent instances are generated using different random seeds. This supports averaging of results and reduces the influence of instance-specific variation, which is particularly important for stochastic solvers like SA and SQA.

For the generation of our problem instances to be efficient graph generation was implemented using an edge-skipping method. This avoids explicitly checking all n^2 possible edges, instead sampling gaps between successive edges using a geometric distribution. This technique uses

the algorithm of Batagelj and Brandes [32], which enables generation of our graphs in expected time proportional to the number of edges rather than the number of possible edges, this speedup was essential for constructing the largest instances in the dataset within practical runtime limits.

All graphs are stored as text files of edge lists (i, j), with zero-indexed node labels and no weights which imply unit edge weights. This representation is compatible with all solvers used and avoids unnecessary memory overhead, a key consideration when working within the constraints of the Google Colab environment.

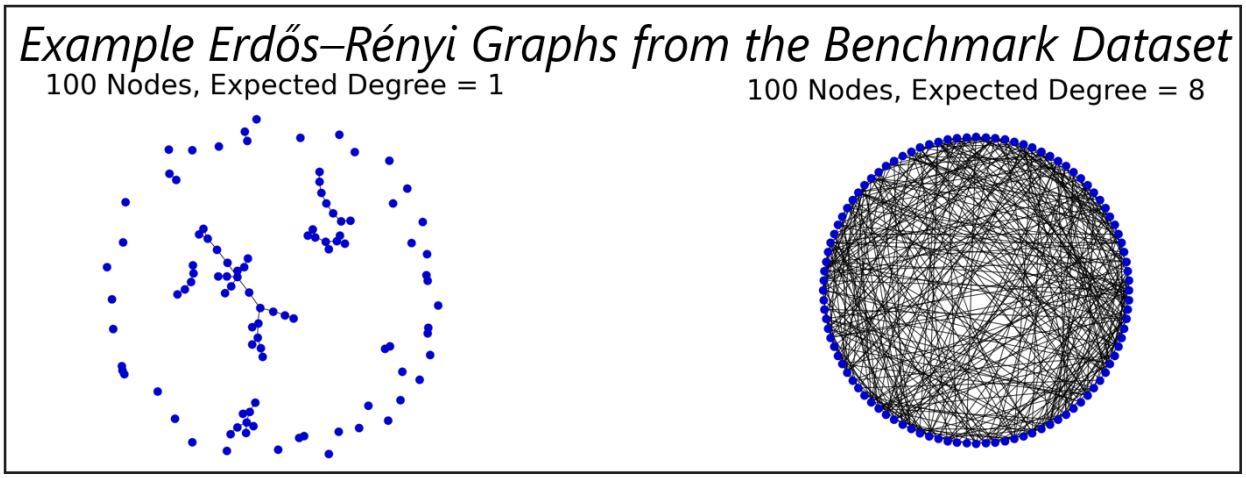


Fig 3.1: Example Erdős-Rényi graphs from the dataset. (Left), sparse instance 100_1.0_0.txt. (Right), dense instance 100_8_0.txt

3.2 Graph Regimes

The dataset is structured into clearly defined graph regimes based on both problem size and expected degree. Node counts were selected as:

$n \in \{100, 1{,}000, 10{,}000, 100{,}000, 1{,}000{,}000\}$, covering five orders of magnitude.

Expected degree values for each of the node sizes are:

$d \in \{0.6, 0.8, 1.0, 1.6, 2, 3, 5, 8, 13, 20\}$, allowing systematic control of graph density.

For each pair, 10 independent instances were generated, giving 500 problem instances in total.

The corresponding density $p = d/(n-1)$ decreases as n increases for fixed d. All graphs remain relatively sparse in a global sense but differ significantly in connectivity. For example, at $n = 100$, the range of d produces graphs from very sparse ($p \approx 0.006$) to moderately dense ($p \approx 0.2$), while at $n = 1{,}000{,}000$, the same d values correspond to extremely sparse graphs ($p \approx 10^{-6}$ to 2×10^{-5}). Despite this, the expected number of edges scales as $O(nd)$, so absolute problem size increases significantly with n.

Three practical regimes emerge from this construction. First, small instances ($n = 100$ and $1{,}000$), which allow all solvers to be applied with minimal resource constraints. Second, medium instances ($n = 10{,}000$) represent a transition point where exact methods start to encounter scalability limits, while heuristic methods remain tractable. Finally, large instances ($n = 100{,}000$ and $1{,}000{,}000$) are dominated by runtime and memory considerations, restricting the use of exact solvers and emphasising the performance of heuristic methods.

Varying d within each size category introduces additional structure. Lower values ($d \leq 2$) produce very sparse graphs with less interaction between variables, while higher values ($d \geq 8$) produce more interconnected graphs with increased interaction. This enables benchmarking to capture how solver performance varies with scale and also complexity, thus giving a more holistic picture of algorithm behaviour across different problem regimes.

3.3 Evaluation Metrics

Solver performance was evaluated using two metrics: objective value and runtime. The objective value measures the quality of the solution, defined as the number of edges cut in the Max-Cut formulation. Runtime measures the computational cost of getting the solution.

Since raw objective values increase with both graph size and expected degree, direct comparison across different regimes is pointless. Instead, solution quality was assessed using a relative performance metric per instance class. For a given instance, the objective value

obtained by a solver is normalised by the best objective value found across all methods for that instance:

$$\text{Relative performance} = \frac{\text{solver's objective}}{\text{best objective in class}}$$

This ensures performance is evaluated consistently across different graph sizes and densities. A value closer to 1 indicates that the solution is closer to the best objective value achieved within that instance class. Results were averaged over all 10 instances within each class to reduce the effect of randomness, in particular for stochastic methods. Together, these metrics provide a balanced assessment of solution quality and computational efficiency.

Chapter 4

Methods

4.1 General Benchmarking Pipeline

The benchmarking process follows a structured pipeline to ensure a fair comparison for all methods. The workflow begins with the input of Erdős-Rényi graph instances generated as described in Chapter 3 above. Each instance is expressed in a solver-appropriate formulation: QUBO and Ising for the annealing methods, mixed-integer quadratic programming (MIQP) for Gurobi, and a constraint programming formulation for CP-SAT. This ensures compatibility with each solver while maintaining equivalence of the optimisation problem.

All four solvers: Simulated Annealing, Simulated Quantum Annealing, Gurobi MIQP, and CP-SAT, are applied independently to each graph instance. Where applicable, solver settings such as random seeds and runtime limits are controlled to maintain consistency.

The main results of each solver are the objective value obtained and the runtime required to reach that solution. Those results are processed to compute relative performance and aggregated across all instances of each graph regime.

Finally, results are sorted by problem size and graph density to facilitate comparison of solution quality, runtime and scalability among methods.

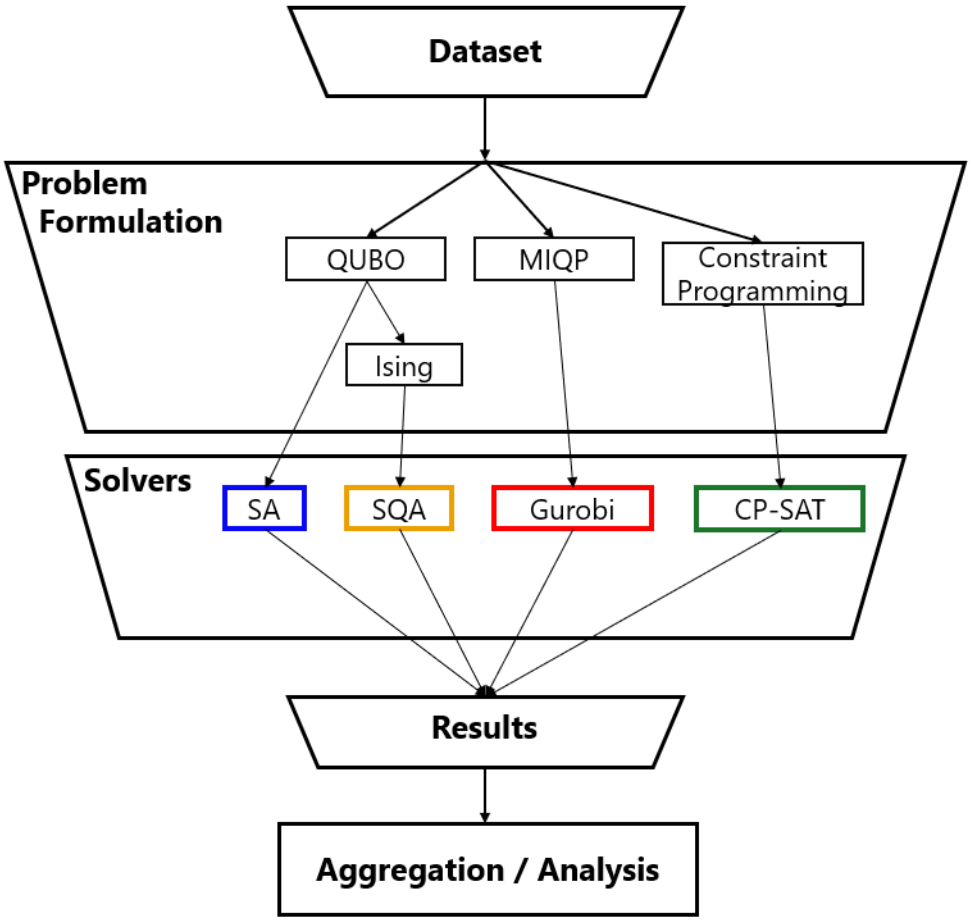


Fig 4.1: Overview of the pipeline of the benchmarking process

4.2 Preliminary Investigations

Early investigations into the baseline performance characteristics of Simulated Annealing (SA) and Simulated Quantum Annealing (SQA) were conducted before the full benchmarking studies. These experiments applied both methods, implemented using D-Wave's Ocean SDK samplers[33], to a set of Max-Cut instances formulated as QUBO problems with problem sizes ranging from 1,000 to 90,000 variables, the dataset used was not generated but rather open-source instances[34].

Number of Vertices	Number of edges	Number of Problem Instances	Weighting Range	Density
1000	2,964	15	{-1000,+1000}	5.93×10^{-3}
10,000	29,694	15	{-1000,+1000}	5.93×10^{-4}
20,000	59,394	15	{-1000,+1000}	2.97×10^{-4}
40,000	118,794	15	{-1000,+1000}	1.48×10^{-4}
90,000	267,294	14	{-1000,+1000}	6.6×10^{-5}

Table 4.1 summary of the open-source dataset used in the preliminary benchmarking tests

The focus at this stage was on finding trends in solution quality and runtime, as well as practical limitations regarding scalability.

The below results showed that for smaller instances, both methods produced quick solutions, but SA consistently produced better objective values than SQA. Different behaviour during runtime emerged as the problem size increased. The runtime of SA greatly increased with problem size, whereas SQA was relatively efficient. For the largest instances tested, SQA was about five times faster than SA and produced solutions of comparable quality.

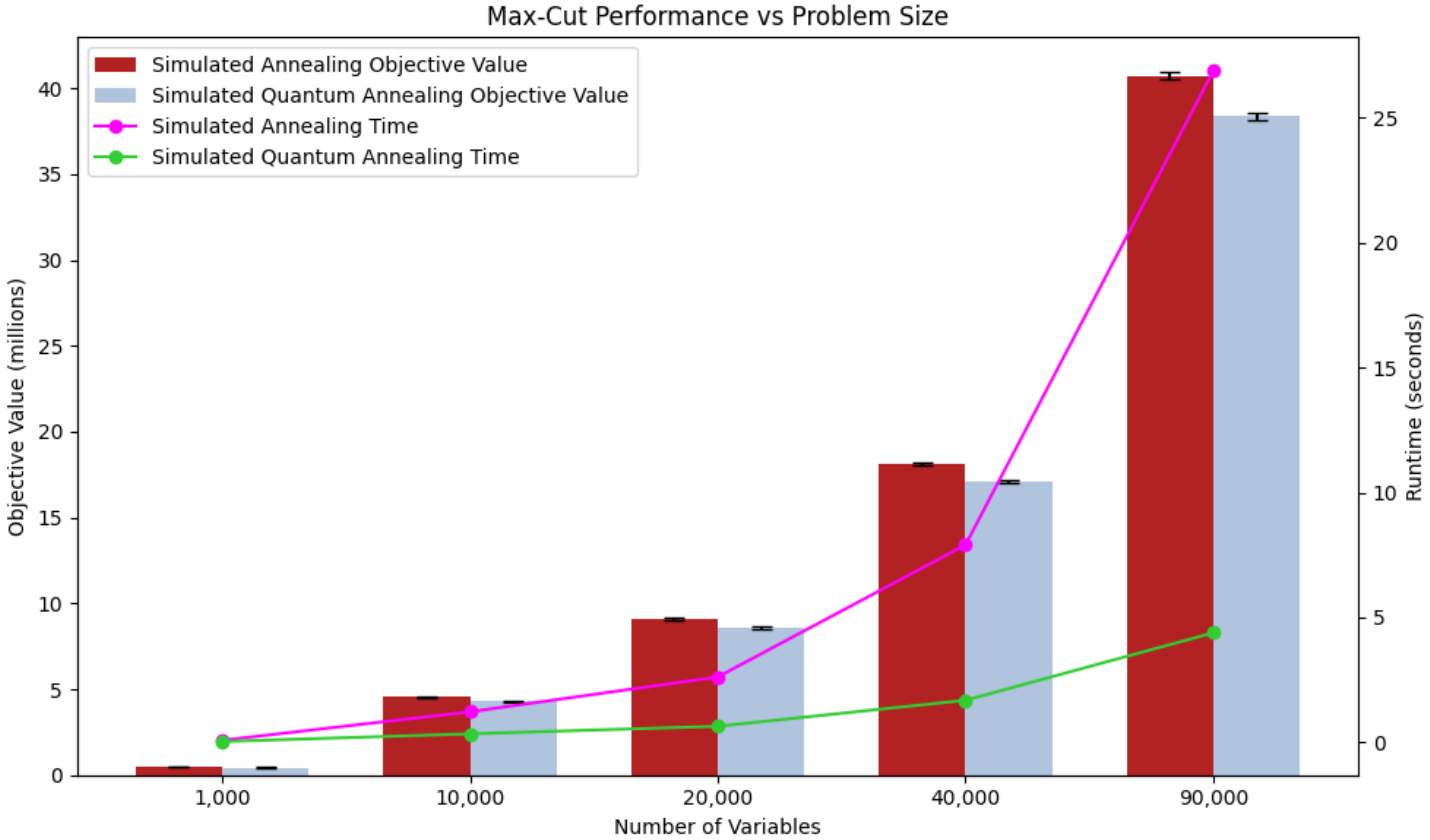


Fig 4.2: Plot of the performance results of SA and SQA solvers for the preliminary tests

This highlights the difficulties of evaluating solver performance in isolation, as neither method dominated across both quality and runtime. This motivated the inclusion of additional classical reference methods for the main benchmarking exploration. Exact approaches such as Gurobi's MIQP and Google OR-Tools CP-SAT provide strong benchmarks for solution quality and allow the performance of heuristic methods to be assessed in relation to high quality solutions. This is particularly important when scaling to larger instances, where differences in objective values become harder to interpret without a reliable reference.

Note that the SQA implementation used in these preliminary experiments was very primitive and did not incorporate any hardware related constraints such as limited qubit connectivity or minor embedding. This means that SQA results only reflect a classical approximation rather than the true performance of a physical quantum annealer, which motivated the modification of the SQA solver to include these hardware constraints in the future.

These preliminary results informed the design of the full benchmarking study by motivating the inclusion of larger and more complex problem sizes, additional solver types, and evaluation metrics that capture relative quality.

4.3 Compute Environment

The benchmarking was conducted in Google Colab notebooks, which provides a virtual Python environment with limited computational resources. This platform was selected for its ease of use, compatibility with the required optimisation libraries, and reproducibility. However, it imposes constraints on memory (12Gb), CPU performance (1 core) and maximum runtime per session (12 hours).

These constraints had a direct influence on the experimental design. Problem instances were handled in batches to avoid memory limits and solver configurations were chosen to keep

runtimes within practical bounds. For the largest instances of 1,000,000 nodes and high expected degree, CP-SAT ran into memory limits and was unable to solve these large problems, this is likely due to the way the problem had to be encoded for its constraint programming formulation.

All implementations were developed using Python, using standard scientific and optimisation libraries. A consistent compute environment guarantees that performance comparisons between solvers reflect algorithmic behaviour under the same practical constraints.

4.4 Solver Implementations

The following section details the implementation of the four optimisation methods to be benchmarked. Each solver is applied to the same dataset of Max-Cut instances but using a formulation appropriate to its underlying framework like QUBO, MIQP, and constraint programming. The focus is on practical implementation choices including solver configuration, parameter choices, and output collection.

4.4.1 Simulated Annealing

Simulated Annealing was implemented using the D-Wave Ocean SDK, where each Max-Cut instance was first represented as a QUBO problem and then converted to a Binary Quadratic Model (BQM). The solver was applied using the `SimulatedAnnealingSampler`, which performs stochastic sampling of the solution space based on a temperature-based acceptance criterion. The solver returns the best solution found along with the objective value and runtime for each instance.

For reproducibility,Sim a fixed random seed was used across all runs. To ensure fair and consistent benchmarking across problem sizes, the parameters for SA were not selected arbitrarily or left as their default values. Instead, a structured parameter exploration was performed using a Taguchi array based experimental design. A large-scale instance

(100,000_3_0.txt) was used as a calibration benchmark. This approach allowed multiple combinations of important parameters, such as temperature schedule characteristics, number of reads and number of sweeps, to be evaluated efficiently without an exhaustive search of all possible combinations. The selected parameter configuration was then fixed for all subsequent experiments to maintain consistency.

A key methodological consideration here is runtime measurement. Runtime was measured externally instead of using the solver-reported execution times, using Phhython timing functions around the full solver call. This covers both model construction time and sampling time. This is important because overhead for problem setup might be excluded by internal solver timing, leading to inconsistent comparisons between different solvers.

The output of the solver contains the best energy (objective value) obtained across all samples and the total runtime for the instance. Though SA typically produces good solutions for small instances, its performance is sensitive to parameter choice and generally shows increasing runtime for larger problem sizes. Hence, it is a strong heuristic baseline for benchmarking studies, notably for assessing solution quality in comparison with other methods.

4.4.2 Simulated Quantum Annealing

Simulated Quantum Annealing was included in this study as a quantum-inspired benchmark. SQA was treated as a classical approximation of quantum annealing based optimisation rather than as evidence of a quantum advantage. Its purpose was to test whether quantum-inspired methods could provide useful solution quality or runtime improvements on the Max-Cut benchmark under the same practical compute limits applied to the other methods.

Each problem instance was read from the edge-list text file and converted to an unweighted Max-Cut representation. From this, an adjacency structure was constructed and the graph

size, edge count, and average degree were extracted. The implementation then applied a hardware-like SQA procedure through the function `SQA_solve_hardware_like_v2`, while a wrapper function `SQA_auto` was used to automatically select decomposition settings and parameters automatically according to instance size and expected degree. This allowed the same solver framework to be applied across the benchmark set while being flexible and adapting the search effort to larger and denser graphs.

Unlike the simpler SQA implementation used in preliminary investigations, the final method did not attempt to solve the full problem in one step. Rather, the graph was decomposed into smaller subproblems, called tiles, which were solved sequentially and used to update the global solution. The tile size, tile pool size, reuse cycles, iteration budget, patience limit, and refresh frequency were selected automatically by `SQA_auto` according to the detected problem regime. Sparse and dense instances were treated differently, and for larger problems, the solver used smaller subproblems with increased iteration budgets to keep within Google Colab memory and runtime limits.

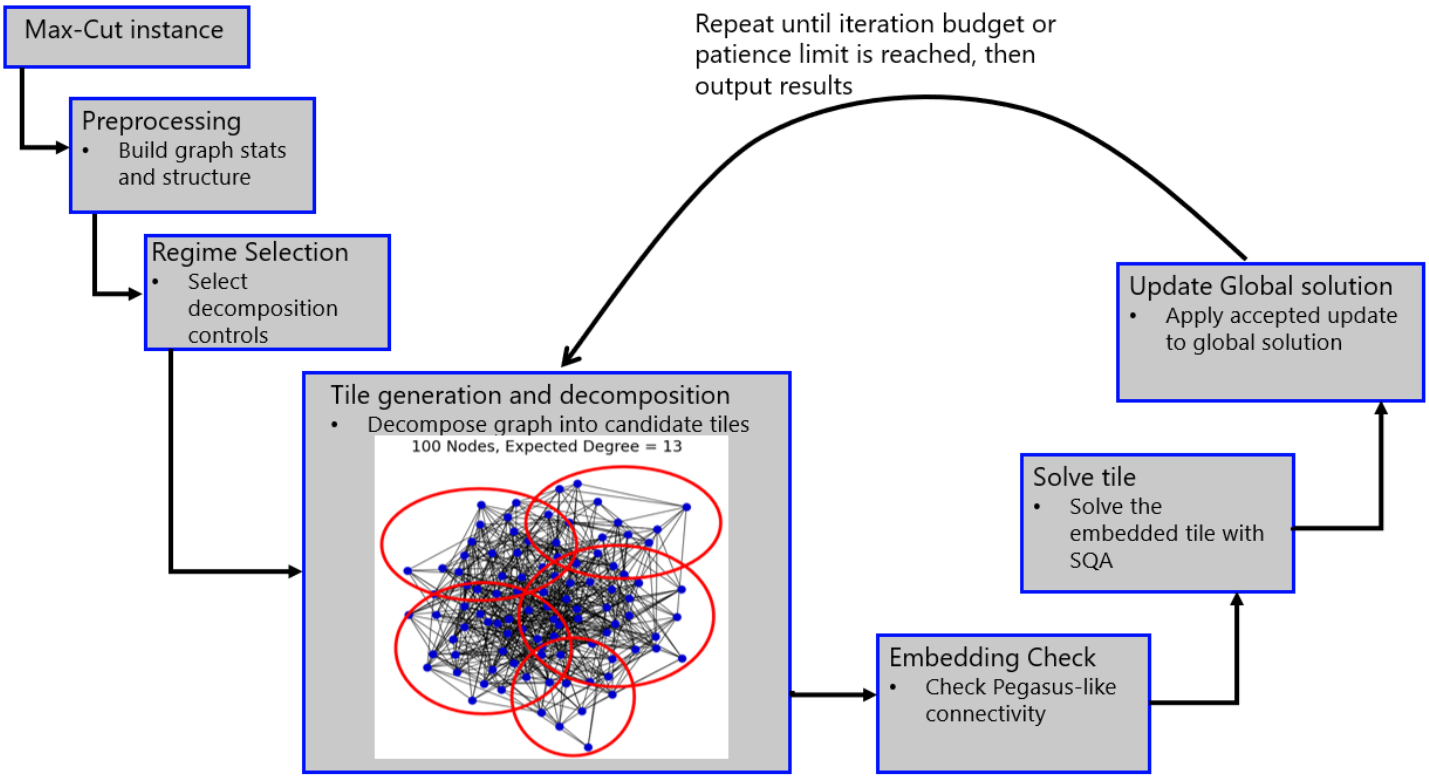


Fig 4.3: Workflow of the SQA solver implementation used for benchmarking

Hardware-like constraints were imposed by requiring each tile to be embeddable onto a Pegasus connectivity graph (shown in Fig 2.5) generated using `dnx.pegasus_graph(pegasus_m)` from the SDK, with `pegasus_m=16` used by default. This does not correspond to execution on physical quantum hardware, but it introduces connectivity restrictions intended to approximate a quantum annealer's deployment more closely than direct SQA. For each candidate tile, an embedding was attempted using `find_embedding`, with only tiles with valid embeddings retained for solution. The embedded binary quadratic model was then sampled using D-Wave Ocean SDK's `RotorModelAnnealingSampler`. Chain strength, chain-break handling, sweep count, beta range, transverse field, and random seed were explicitly controlled, with a fixed seed used for reproducibility.

Similar to the SA implementation, parameter selection was not left entirely to default settings. A structured calibration stage was carried out on a representative benchmark instance, and the instance 1000_3_4.txt was used for this purpose in the SQA case as its small size allowed for quick testing. This allowed key solver parameters to be tested in a controlled way before the main benchmark runs so as to maximise the potential of the SQA step. The selected configuration was then fixed for the wider experiments to support consistency.

The tile objective included both internal Max-Cut interactions and boundary terms representing connections from the tile to the rest of the graph. These boundary terms were adaptively scaled so that local subproblem solutions were still influenced by the current global assignment. A warm start mechanism was also used, which allowed the current global state to be passed to the sampler as an initial state. After each tile was solved, the resulting local update was accepted if it improved the cut value, and the global solution was updated accordingly.

For benchmarking, the solver recorded the best objective found, solver wall time, embedding time, average chain length and number of iterations completed. This provided a more realistic indication of the overheads associated with hardware-constrained annealing-style optimisation than direct unconstrained SQA alone.

4.4.3 Gurobi MIQP

Gurobi was included as a high-performance commercial exact solver. Unlike SA and SQA, which are treated as heuristic methods, the Gurobi implementation solved Max-Cut using a general MIQP framework. This was useful as a reference for solution quality on smaller and medium-sized instances and showed how quickly exact optimisation is constrained by runtime as problem size increases.

The Gurobi MIQP model is closely related to the QUBO representation used elsewhere. A QUBO can be viewed as a special case of MIQP, since it consists of binary decision variables and a quadratic objective with no explicit constraints. In this case, the Max-Cut problem was written directly as a binary quadratic objective and passed to Gurobi as an MIQP model through their Python API. This allowed us to keep the same underlying quadratic structure of the annealing-based methods while solving it with a general-purpose exact optimiser. Gurobi treats models with integer variables and quadratic objectives as MIQP instances and can solve them using branch-and-bound, cutting planes, and primal heuristics.

The solver read each problem instance, constructed the MIQP model and optimised it using the Gurobi Python interface. A time limit was imposed through the `TimeLimit` parameter. To encourage strong feasible solutions early in the solve, heuristic targeted settings were used, including `MIPFocus = 1`, a high heuristics level and a set minimum heuristic time. These settings are aligned with Gurobi's documented parameters for prioritising incumbent solution quality under restricted solve times[35].

A practical time limit was necessary because exact MIQP methods consume more runtime on large combinatorial instances. Gurobi was treated as an exact benchmark with a bounded runtime rather than as a method expected to prove optimality on the largest graphs. The solver recorded final objective, runtime, best bound, optimality gap, and optimality status, allowing both solution quality and proof progress to be compared fairly across instances.

4.4.4 CP-SAT

Google OR-Tools CP-SAT was included as an open-source exact reference method based on constraint programming rather than quadratic programming. CP-SAT is designed for integer optimisation models[36] and combines constraint programming with SAT-based search and propagation.

To apply CP-SAT to our problem instances, each Max-Cut instance was reformulated from the graph representation into a Boolean constraint model. As before a binary variable $x_i \in \{0,1\}$ was assigned to each vertex $i \in V$, indicating the side of the cut. But because CP-SAT does not solve the problem as a quadratic binary model directly, an additional Boolean variable $y_{ij} \in \{0,1\}$ was needed for each edge $(i,j) \in E$, where $y_{ij} = 1$ if the edge is cut and $y_{ij} = 0$ otherwise. The objective can therefore be written as:

$$max \sum_{(i,j)\in E} y_{ij}.$$

The XOR relationship $y_{ij} = x_i \oplus x_j$ could be enforced using these linear inequalities:

$$y_{ij} \leq x_i + x_j,$$
$$y_{ij} \leq 2 - (x_i + x_j),$$
$$y_{ij} \geq x_i - x_j,$$
$$y_{ij} \geq x_j - x_i.$$

This produced a model where each instance has $n + m$ binary variables and $4m$ linear constraints for each instance, where n and m are the number of nodes and edges respectively. This formulation is larger than the direct QUBO or MIQP representation due to the constraints needed, but it allows the problem to be handled within the integer modelling framework required by CP-SAT.

The implementation read each edge-list instance, built the CP-SAT model in Python, and solved it using `CpSolver`. A practical time limit was imposed through `max_time_in_seconds` identical to that given to Gurobi, and the number of search workers was controlled using `num_search_workers` to keep to the memory constraints of Google Colab. The solver recorded the objective value, runtime, best bound, and termination status. As with Gurobi, CP-SAT was treated as an exact benchmark under bounded runtime rather than a method expected to certify optimality on the largest instances.

4.5 Solution Strategy

Because the full benchmark dataset contained lots of graph sizes, expected degrees, and instances, the dataset was not solved in a single run. It adopted a batch-based approach instead. This was needed for both practical management of experiments and also to stay within Google Colab's runtime and memory limits.

A batch solving function was used to organise solver execution by node size and method. For a selected node size, the function first collected all matching instance files and then assigned the appropriate solver interface depending on whether SA, SQA, Gurobi, or CP-SAT was being evaluated. Time limits and worker settings were passed only to the methods that needed them and solver-specific metadata such as embedding time or iteration count were recorded where available.

To avoid unnecessary reruns, the strategy maintained a raw results CSV file stored in Google Drive, containing previously completed runs. Before each batch, the function checked this file and skipped instances that had already been solved. This enabled full dataset completion over multiple sessions. It also permits partial execution, solving only the next unfinished subset of instances. This was useful for the larger, slower exact methods like CP-SAT that often crashed due to hitting memory limits.

After each run, the solver output was written to a common CSV structure containing solver name, graph size, expected degree, instance number, objective value, runtime, and other method-specific diagnostics. This ensured that all methods could later be aggregated and analysed using a consistent data pipeline.

4.6 Aggregation and Visualisation of Results

The recorded solver outputs were grouped by method, node count and expected degree for post-processing. For each combination of node count and expected degree, the ten corresponding graph instances were aggregated to get summary performance statistics for each solver.

Mean objective value and mean runtime were used as the main summary statistics. These were chosen to show typical solver behaviour while reducing the influence of variation between individual random graph instances.

Objective value and runtime were plotted against expected degree separately for each node size, allowing density effects to be examined without overcrowding the figures. Relative performance heatmaps were also produced for sparse and dense regimes which provide a compact summary of how solver suitability changes across the dataset. Those visualisations form the basis of the comparative analysis seen in Chapter 5.

Chapter 5

Results

5.1 Overview of Benchmarks

Section 5 presents the benchmarking results of the four Max-Cut solution methods over the full dataset of Erdős-Rényi graphs. The results are organised by problem size to show how solver behaviour changes as the number of nodes increases, and within each node size category, performance is evaluated using objective value and runtime, examined as functions of expected degree of the instances.

The first part of the chapter uses objective versus degree and runtime versus degree plots to show how each method performs across sparse and dense instances at each node size. This is followed by an aggregated comparison using heatmaps, which highlight relative method performance across different density regimes. The final sections draw together these results to identify trends in solution quality, scalability, and computational cost. The emphasis is on fair empirical comparisons under a common benchmark setting rather than on claiming general solver superiority.

5.2 Results on Small Instances

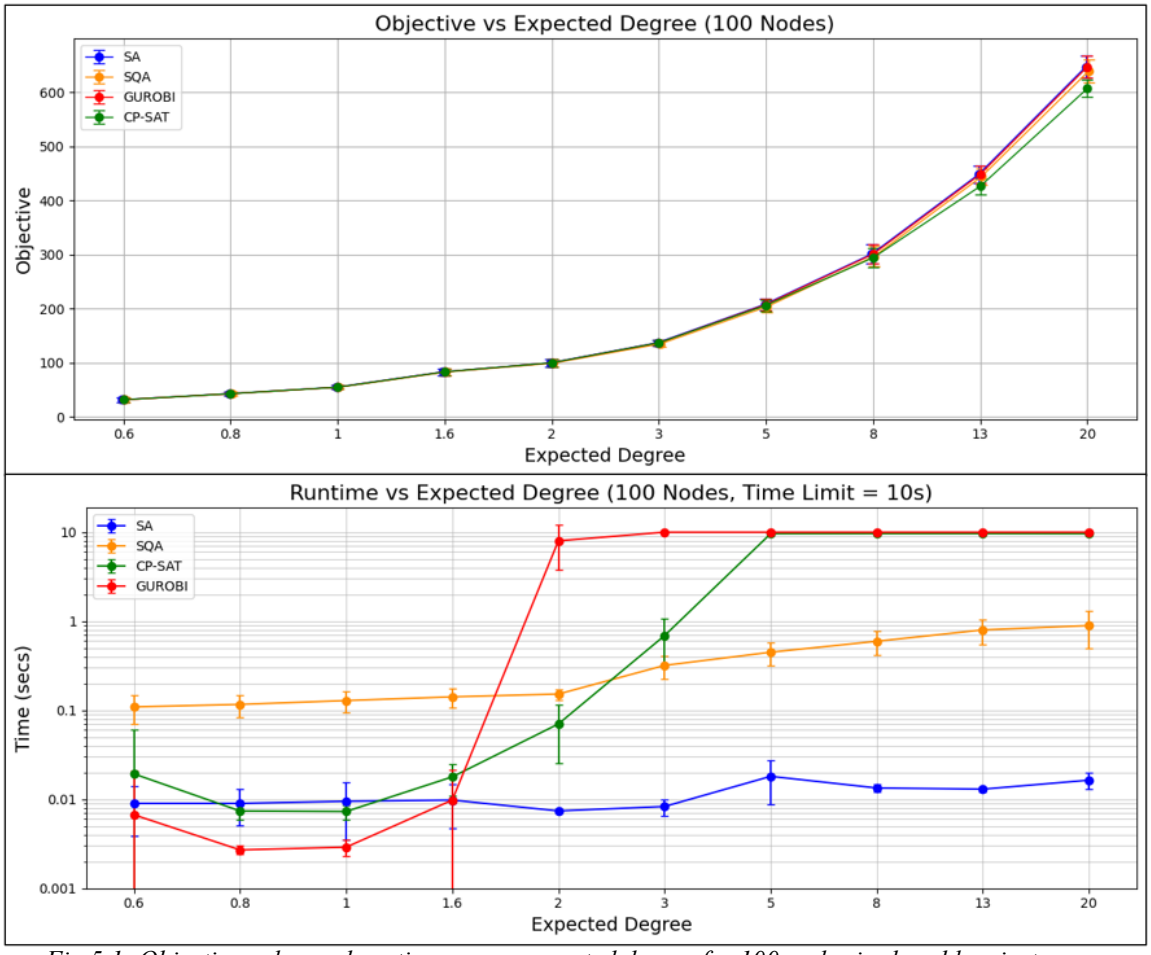


Fig 5.1: Objective value and runtime versus expected degree for 100 node sized problem instances.

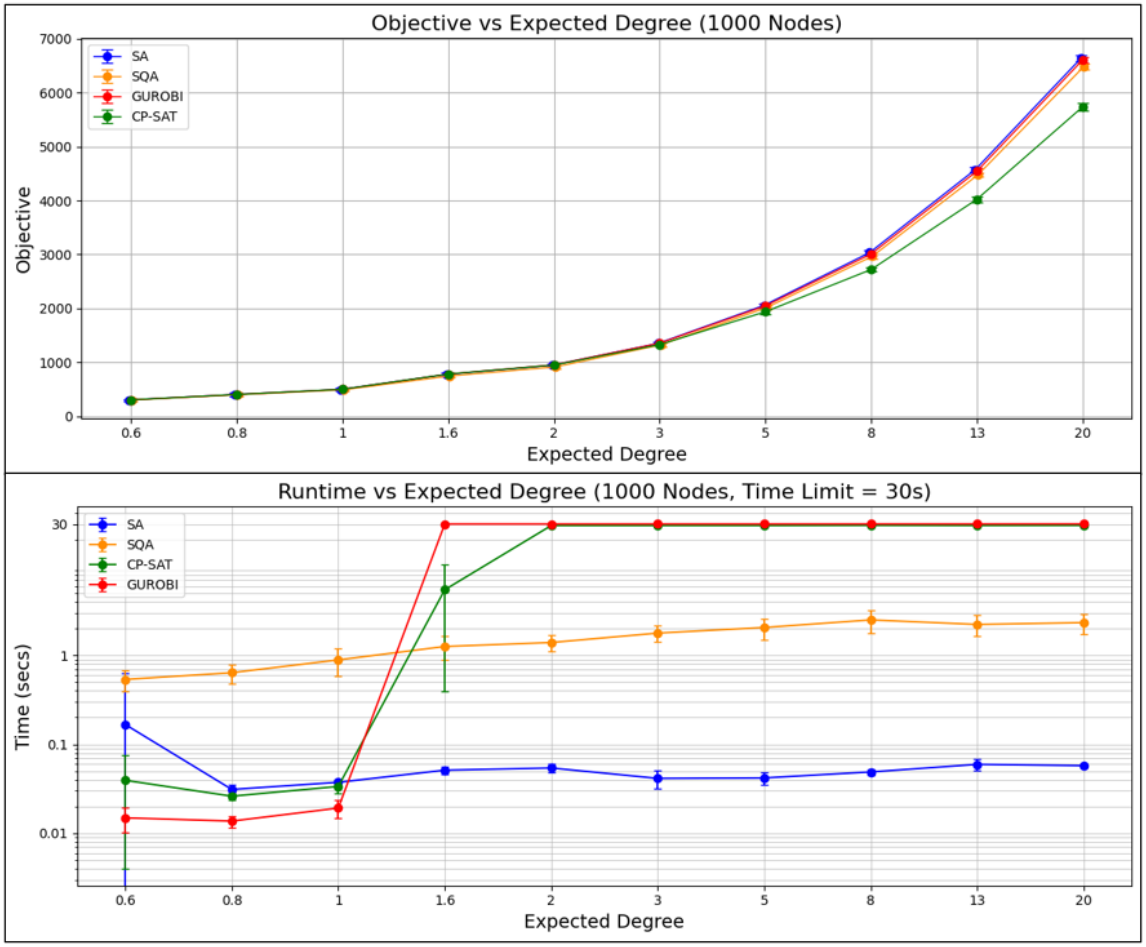


Fig 5.2: Objective value and runtime versus expected degree for 1,000 node sized problem instances

For the small instances (100 and 1,000 nodes), all four approaches show fairly similar objective values at low expected degrees, with better differentiation only when the overall density of the instance increases. SA, SQA and Gurobi remained closely grouped across most of the degree range, while CP-SAT fell behind on the densest cases, for both node sizes.

The runtime plots present a much clearer picture of the structural shift. For both sizes, runtimes for the exact methods rose rapidly once the expected degree moved above 1, after which Gurobi and CP-SAT started to hit their time limits. This aligns with the Erdős-Rényi giant-component threshold, when the average degree goes above 1, a unique giant connected component forms with a high probability, making the instances much less fragmented and harder to decompose into small independent parts. SA remained the fastest method, while SQA was slower but still competitive across the full range.

5.3 Results on Medium-sized Instances

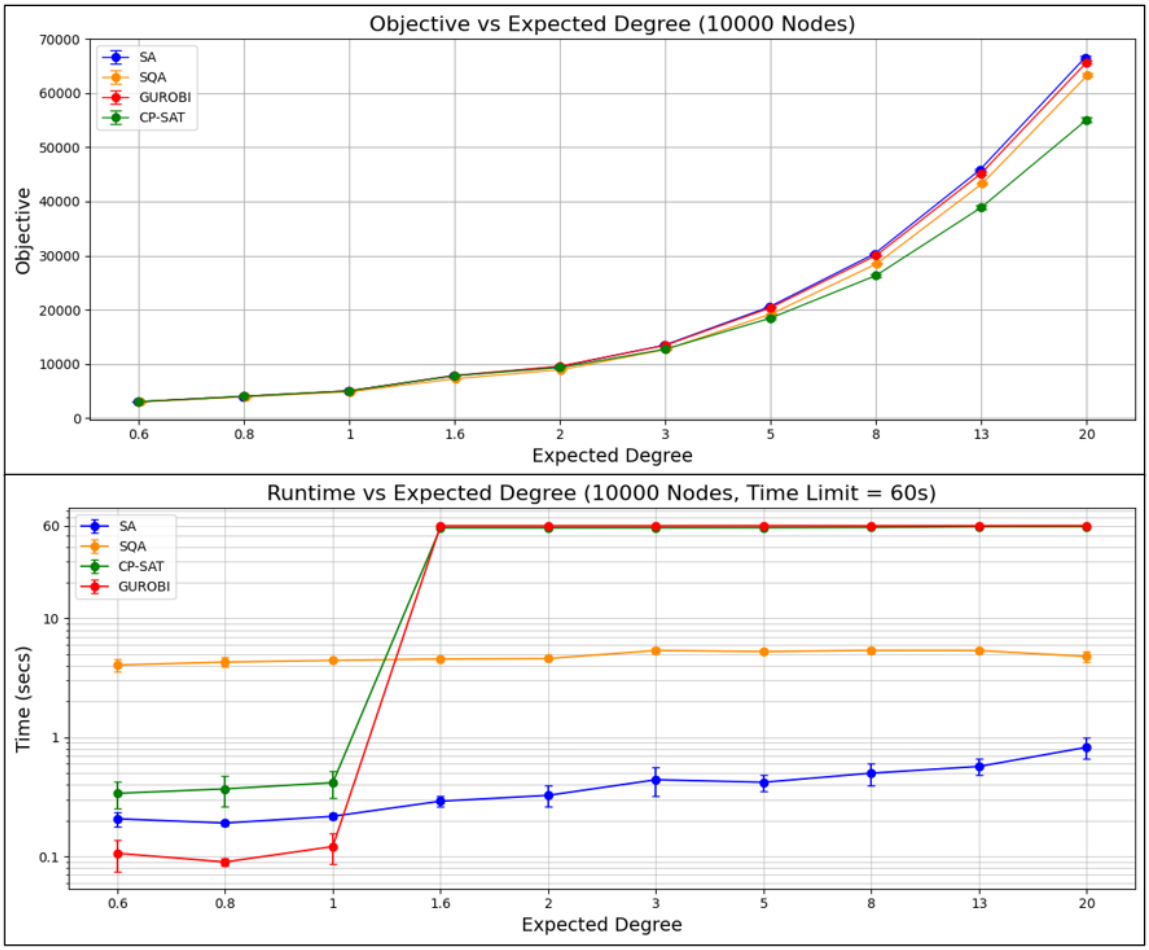


Fig 5.3: Objective value and runtime versus expected degree for 10,000 node sized problem instances.

For the medium-sized instances (10,000 nodes), the difference in the objectives among the different methods starts to become apparent. SA and Gurobi consistently found better objectives than the other two approaches. CP-SAT's results fell further behind on the densest instances, showing a change in solution quality in addition to runtime for larger sized graphs.

Runtime results showed a clearer separation. Gurobi and CP-SAT hit the 60 second time limit from expected degree 1.6 onward, while SA and SQA remained competitive across the full range. This jump in runtime beyond expected degree 1 is consistent with the Erdős-Rényi giant-component threshold mentioned before. At this scale, SA was still the fastest method and SQA remained practical, while the exact methods scaled worse after the sparse instances.

5.4 Results on Large Instances

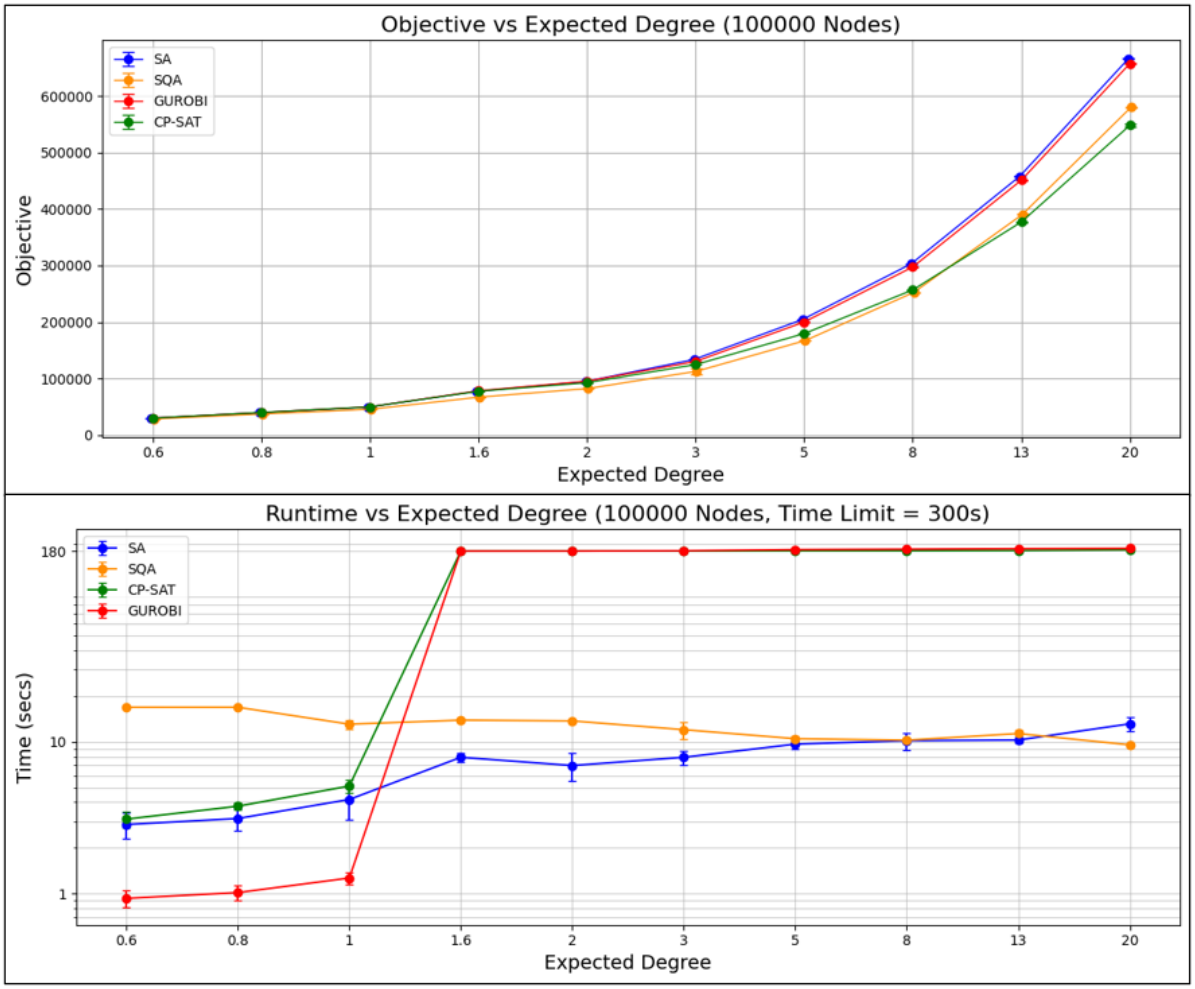


Fig 5.4: Objective value and runtime versus expected degree for 100,000 node sized problem instances

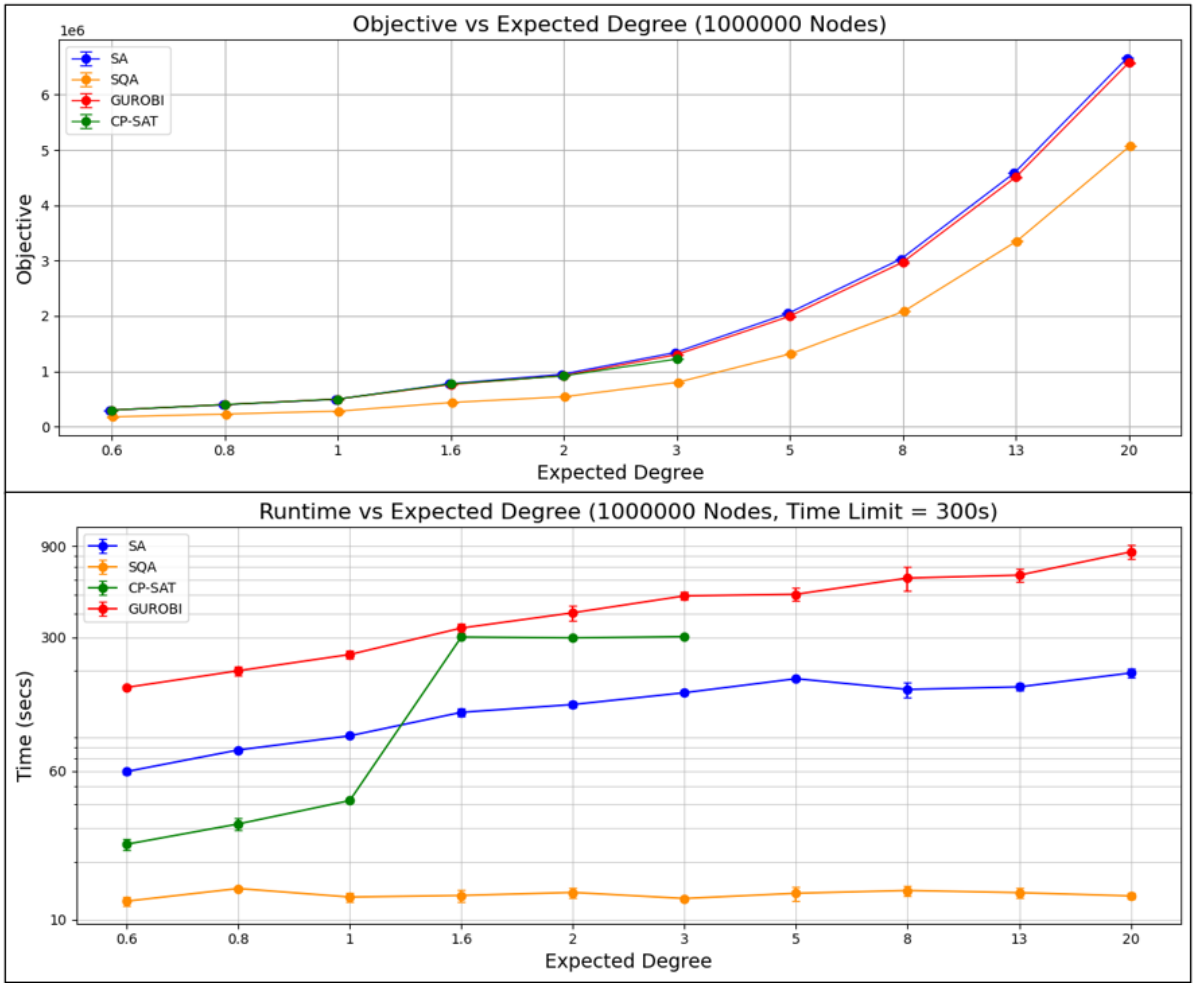


Fig 5.5: Objective value and runtime versus expected degree for 1,000,000 node sized problem instances

For the large instances (100,000 and 1,000,000 nodes), the benchmark became primarily a test of scalability. At 100,000 nodes, both SA and Gurobi continued to achieve the strongest objective values, while SQA showed falloff in quality as well as CP-SAT. At 1,000,000 nodes, this separation became much larger. SA and Gurobi remained closely matched in objective value across the full degree range. SQA showed a substantial decline as expected degree increased. CP-SAT results were only available for the sparser part of the range, reflecting practical solver limits at this scale.

The runtime results showed an equally clear change in method suitability. At 100,000 nodes, Gurobi and CP-SAT hit the time limit immediately after the expected degree moved above 1, again consistent with the giant-component threshold. At 1,000,000 nodes, SQA became the fastest method by a large margin and its runtime remained almost flat across degrees, but this

came with reduced objective performance. SA remained usable but grew more computationally expensive, while the exact methods were no longer practical for broad benchmarking under the available compute limits.

5.5 Effects of Graph Density

Looking at the previous results, graph density has emerged as one of the main factors shaping solver behaviour across the benchmark. While increasing expected degree naturally increased objective value by creating more edges to cut, it also changed the complexity of the instances and therefore the performance differences between methods. In the sparsest cases, performance was often similar across solvers, particularly for smaller graphs where exact methods remained competitive. As the expected degree increased, differences in runtime and solution quality between methods became more apparent.

However, this effect was not uniform across methods. SA and Gurobi generally retained the best objective values as density increased, whereas SQA showed a larger quality loss on the largest dense instances. CP-SAT also became less competitive as density and graph size increased together. Runtime patterns showed an even clearer split, especially around expected degree of 1 and above, where the exact methods often hit their runtime limits. For this reason, the next section compares the methods at an aggregated level by separating the results into sparse ($d \leq 1$) and dense regimes ($d \geq 2$), allowing these trends to be examined more clearly across the full benchmark set.

5.6 Aggregated Comparison of Methods

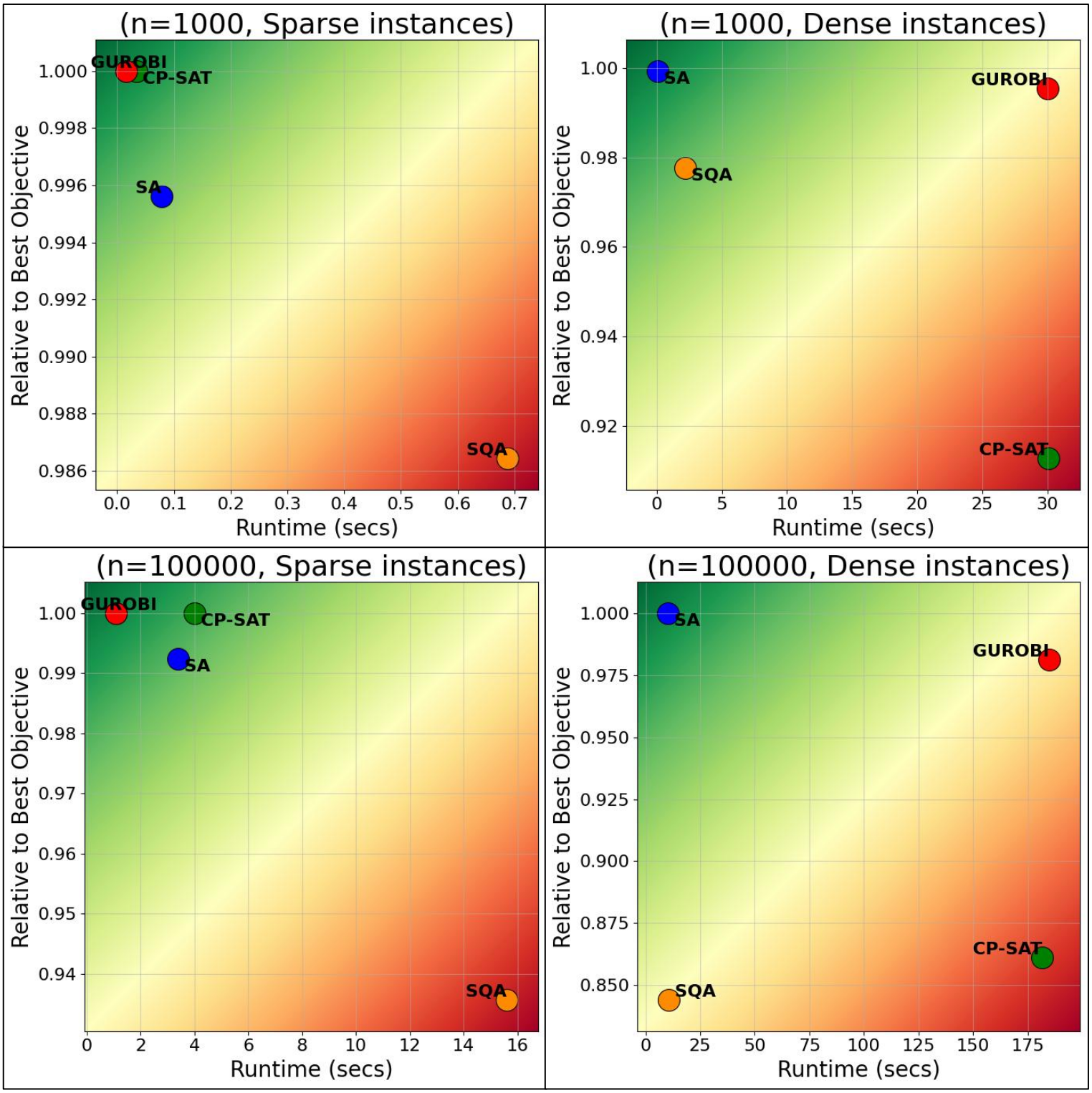


Fig 5.6: Heatmaps of solver performance for sparse (expected degree ≤ 1) and dense (expected degree ≥ 2) graph classes at 1,000 and 100,000 nodes.

The above figure provides an aggregated comparison of each method's performance by separating the dataset into sparse and dense regimes, then plotting runtime against objective value relative to the best method for each case. This view makes the trade-off between solution quality and computational cost easier to compare across problem scales.

In the sparse cases, Gurobi and CP-SAT performed the best, often having the best objectives and comparable runtimes to SA. SA performed well in the sparse instances with competitive runtimes and objective values compared to the exact methods. SQA performed worst in these instances with longer runtimes and worse objective values.

Dense instance plots showed a better separation between methods. SA became the strongest method, with the best objectives and lowest runtimes throughout. Gurobi still achieved high quality solutions, but with a substantial increase in runtime compared to the sparse instances. CP-SAT and especially SQA showed a larger deterioration in objective quality on dense large instances. However, SQA saw a substantial improvement in its runtimes compared to those of the exact methods. Overall, the heatmaps confirm the advantages of more scalable methods but potentially hint at SQA being comparatively less affected by increased problem complexity.

5.7 Technical Discussion and Conclusions

The results show that solver performance heavily depended on graph size and density. Simulated annealing was the best performing method overall; it repeatedly achieved best or near-best objective values while having low runtimes across a wide range of instance sizes. This made it the strongest all-round solver under the benchmark conditions used. Simulated quantum annealing was less competitive on objective value for sparse instances but performed much better at very large scales as its runtime remained comparatively stable even when the instances became computationally expensive for exact methods. SA's dominance is likely down to the energy landscapes of unweighted Max-Cut problems being much simpler

than that of weighted Max-Cut, this means SA can escape local minima more easily and reach better final solutions. Max-Cut's simple QUBO formulation also contributed to this, on more complex problems SA may not perform so well.

The exact methods reveal a different trade-off. Gurobi achieved objective values that matched or nearly matched the best observed results for all regimes. CP-SAT also remained competitive on sparse problems only. However, both exact approaches scaled less effectively as graph size and complexity increased. In practice, their performance was limited by runtime, especially once the expected degree moved above 1. This behaviour is consistent with the Erdős–Rényi phase transition, where a giant connected component emerges, making the instances exponentially more complex[37].

From a practical perspective, the best solver depends on the intended application. Where objective quality is the main priority and longer runtimes are acceptable, such as in some finance or logistics settings, exact methods may still be worthwhile on smaller instances. Where decisions must be made quickly, such as in routing or repeated large-scale evaluations, SA offers the most balanced performance, while SQA shows promise when runtime at very large scales is valued more than obtaining the very best solution.

A key limitation of these results is that the benchmark instances use unweighted Erdős–Rényi graphs. Such instances are useful for controlled comparison but are very regular and artificial instances which may not represent the structural features of many real-world optimisation problems.

Chapter 6

Future Work, Ethics and Sustainability

6.1 Future Work

A natural next step for this project is to extend the benchmarking framework from simple unweighted Max-Cut to portfolio optimisation (PO). This would test the same solvers on a problem of very different structure. For the Max-Cut problem, complexity was largely controlled by the number of edges in the graph. In PO, difficulty instead arises from the dense quadratic interactions induced by the covariance matrix, so the problem provides a useful contrast to the sparse Erdős–Rényi benchmark instances used before.

The starting point would be the standard mean-variance formulation[38], where we let w_i denote the portfolio weight assigned to asset i, let μ_i denote the expected return, and let Σ denote the covariance matrix. The common objective is to minimize portfolio risk while imposing a required return R, so a quadratic model of the form is given as:

$$minimise\ w^{\top}\Sigma w$$

subject to the constraints:

$$\sum_{i=1}^{N} w_i = 1,$$

$$\sum_{i=1}^{N} \mu_i\, w_i \geq R,$$

$$w_i \geq 0.$$

To make the problem compatible with SA and SQA, the model would then need to be discretised. The most practical approach is a bucketed one-hot encoding in which each asset is assigned one value from a fixed set of allocation levels. With this approach, each

continuous weight w_i is replaced by a number of binary bucket variables, and exactly one bucket is selected for each asset. Converting the model into a QUBO would then require three main steps: first, replacing continuous weights by binary bucket variables; second, rewriting the quadratic variance term in terms of those binary variables; and third, converting the linear constraints, like budget and return constraints, into quadratic penalty terms added to the objective. This would produce a single QUBO formulation that could be used by SQA.

Future work would focus particularly on choosing bucket values and bucket count so that the resulting binary instances are large enough to be informative, but still manageable under Google Colab's environment limits. A moderate bucket count is preferable because problem size grows rapidly with NB, where N is the number of assets and B is the number of buckets, while the number of pairwise interactions is $\binom{NB}{2}$. This provides a useful comparison to the Max-Cut study, where practical complexity was largely tied to the number of edges.

A suitable dataset was identified in the OR-Library portfolio benchmark set, which provides five standard instances with 31, 85, 89, 98 and 225 assets[39]. These instances were introduced for cardinality-constrained portfolio optimisation and have been widely reused as benchmark test cases in later studies. They provide a strong basis for extending the present benchmarking methodology to a denser optimisation problem while linking to the established literature.

6.2 Ethical Considerations

This project focuses entirely on the benchmarking of classical and quantum-inspired optimisation techniques for combinatorial optimisation problems, and does not involve any personal data, human participants, or safety-critical systems. However, some ethical considerations are still relevant, particularly in relation to research integrity and the reporting of results.

One important consideration is the transparent and responsible presentation of benchmarking results. Benchmark studies are sensitive to design choices such as problem formulation, solver parameter settings, and performance metrics. Care was therefore taken to use a consistently generated dataset, consistent formulations and clear evaluation metrics to support a fair comparison between methods. Overstating the performance of any solver could lead to misleading conclusions, particularly in an area where claims around quantum and quantum-inspired optimisation are often interpreted broadly. For this reason, the results of this project are limited to the benchmark instances, methods, parameter settings, and compute environment used in this study.

6.3 Sustainability

From the perspective of sustainability, computational optimisation can involve significant resource use. Large benchmark studies require repeated solver executions, which consumes processing time, memory and energy. In addition, practical quantum annealing hardware relies on energy-intensive cooling systems, even though this project focused on simulation and software-based benchmarking rather than direct hardware evaluation.

The analysis of runtime, scalability and practical solver efficiency contributes to a better understanding of how optimisation problems may be solved more effectively. In this sense, benchmarking can support more efficient use of computational resources by helping identify which methods are most suitable for different problem regimes, and where additional computational cost is or is not justified.

References

[1] A. Lucas, “Ising formulations of many NP problems,” *Frontiers in Physics*, vol. 2, 2014, doi: https://doi.org/10.3389/fphy.2014.00005.

[2] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by Simulated Annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, May 1983, doi: https://doi.org/10.1126/science.220.4598.671.

[3] T. Kadowaki and H. Nishimori, “Quantum annealing in the transverse Ising model,” *Physical Review E*, vol. 58, no. 5, pp. 5355–5363, Nov. 1998, doi: https://doi.org/10.1103/physreve.58.5355.

[4] M. X. Goemans and D. P. Williamson, “Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming,” *Journal of the ACM*, vol. 42, no. 6, pp. 1115–1145, Nov. 1995, doi: https://doi.org/10.1145/227683.227684.

[5] R. M. Karp, “Reducibility among Combinatorial Problems,” *Complexity of Computer Computations*, pp. 85–103, 1972, doi: https://doi.org/10.1007/978-1-4684-2001-2_9 .

[6] J. Vodeb, V. Eržen, T. Hrga, and J. Povh, “Accuracy and Performance Evaluation of Quantum, Classical and Hybrid Solvers for the Max-Cut Problem,” *arXiv.org*, 2024. https://arxiv.org/abs/2412.07460

[7] M.-W. Park and Y.-D. Kim, “A systematic procedure for setting parameters in simulated annealing algorithms,” *Computers & Operations Research*, vol. 25, no. 3, pp. 207–217, Mar. 1998, doi: https://doi.org/10.1016/S0305-0548(97)00054-3.

[8] Y. Lin, Z. Bian, and X. Liu, “Developing a dynamic neighborhood structure for an adaptive hybrid simulated annealing – tabu search algorithm to solve the symmetrical traveling salesman problem,” *Applied Soft Computing*, vol. 49, pp. 937–952, Dec. 2016, doi: https://doi.org/10.1016/j.asoc.2016.08.036.

[9] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, Apr. 1997, doi: https://doi.org/10.1109/4235.585893.

[10] F. Peres and M. Castelli, “Combinatorial Optimization Problems and Metaheuristics: Review, Challenges, Design, and Development,” *Applied Sciences*, vol. 11, no. 14, p. 6449, Jul. 2021, doi: https://doi.org/10.3390/app11146449.

[11] “‘Eureka — You Shrink!,’” *Lecture Notes in Computer Science*, pp. 1–10, 2003, doi: https://doi.org/10.1007/3-540-36478-1_1.

[12] Laurent Perron, Frédéric Didier, and Steven Gay. The CP-SAT-LP Solver (Invited Talk). In 29th International Conference on Principles and Practice of Constraint Programming (CP 2023). Leibniz International Proceedings in Informatics (LIPIcs), Volume 280, pp. 3:1-3:2, Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2023) https://doi.org/10.4230/LIPIcs.CP.2023.3

[13] N. Eén and N. Sörensson, “An Extensible SAT-solver,” *Theory and Applications of Satisfiability Testing*, pp. 502–518, 2004, doi: https://doi.org/10.1007/978-3-540-24605-3_37.

[14] G. E. Santoro, R. Martonak, Erio Tosatti, and R. Car, “Theory of Quantum Annealing of an Ising Spin Glass,” *Science*, vol. 295, no. 5564, pp. 2427–2430, Mar. 2002, doi: https://doi.org/10.1126/science.1068774.

[15] E. Farhi, J. Goldstone, S. Gutmann, J. Lapan, A. Lundgren, and D. Preda, “A Quantum Adiabatic Evolution Algorithm Applied to Random Instances of an NP-Complete Problem,” *Science*, vol. 292, no. 5516, pp. 472–475, Apr. 2001, doi: https://doi.org/10.1126/science.1057726.

[16] https://www.researchgate.net/figure/Quantum-annealing-functions-As-and-Bs-The-function-At-is-defined-as-twice-the_fig13_268391588

[17] K. Boothby, P. Bunyk, J. Raymond, and A. Roy, “Next-Generation Topology of D-Wave Quantum Processors,” *arXiv.org*, 2020. https://arxiv.org/abs/2003.00133

[18] V. Choi, “Minor-Embedding in Adiabatic Quantum Computation: I. The Parameter Setting Problem,” *arXiv.org*, 2026. https://arxiv.org/abs/0804.4884

[19] Sascha Sebastian Wald. Thermalisation and Relaxation of Quantum Systems. General Physics [physics.gen-ph]. Université de Lorraine, 2017. doi: https://theses.hal.science/tel-01743828/

[20] S. Okada, Masayuki Ohzeki, Masayoshi Terabe, and S. Taguchi, “Improving solutions by embedding larger subproblems in a D-Wave quantum annealer,” *Scientific Reports*, vol. 9, no. 1, Feb. 2019, doi: https://doi.org/10.1038/s41598-018-38388-4.

[21] T. F. Rønnow *et al.*, “Defining and detecting quantum speedup,” *Science*, vol. 345, no. 6195, pp. 420–424, Jul. 2014, doi: https://doi.org/10.1126/science.1252319.

[22] D. Battaglia, G. E. Santoro, and Erio Tosatti, “Optimization by quantum annealing: Lessons from hard satisfiability problems,” vol. 71, no. 6, Jun. 2005, doi: https://doi.org/10.1103/physreve.71.066707.

[23] S. Kikuura, R. Igata, Y. Shingu, and S. Watabe, “Performance Benchmarking of Quantum Algorithms for Hard Combinatorial Optimization Problems: A Comparative Study of non-FTQC Approaches,” *arXiv.org*, 2024. https://arxiv.org/abs/2410.22810

[24] T. Bartz-Beielstein *et al.*, “CIplus Band 2/2020 Benchmarking in Optimization: Best Practice and Open Issues.” Available: https://d-nb.info/1214641016/34

[25] J. King, S. Yarkoni, Nevisi, Mayssam M, J. P. Hilton, and C. C. McGeoch, “Benchmarking a quantum annealing processor with the time-to-target metric,” *arXiv.org*, 2026. https://arxiv.org/abs/1508.05087

[26] E. Pelofske, “Comparing three generations of D-Wave quantum annealers for minor embedded combinatorial optimization problems,” *Quantum Science and Technology*, vol. 10, no. 2, p. 025025, Feb. 2025, doi: https://doi.org/10.1088/2058-9565/adb029.

[27] D. Vert, M. Willsch, B. Yenilen, R. Sirdey, S. Louise, and K. Michielsen, “Benchmarking quantum annealing with maximum cardinality matching problems,” *Frontiers in Computer Science*, vol. 6, Jun. 2024, doi: https://doi.org/10.3389/fcomp.2024.1286057.

[28] C. McGeoch, “How NOT to Fool the Masses When Giving Performance Results for Quantum Computers,” *arXiv.org*, 2024. https://arxiv.org/abs/2411.08860

[29] S. Kim, S.-W. Ahn, I.-S. Suh, A. W. Dowling, E. Lee, and T. Luo, “Quantum Annealing for Combinatorial Optimization: A Benchmarking Study,” *arXiv.org*, 2025. https://arxiv.org/abs/2504.06201

[30] F. A. Quinton, P. A. S. Myhr, M. Barani, P. Crespo del Granado, and H. Zhang, “Quantum annealing applications, challenges and limitations for optimisation problems compared to classical solvers,” *Scientific Reports*, vol. 15, no. 1, Apr. 2025, doi: https://doi.org/10.1038/s41598-025-96220-2.

[31] S. A. J, “Optimization Algorithms for Combinatorial Problems: A Comparative Study of Quantum Annealing and Classical Methods,” *International Journal of Intelligent Systems and Applications in Engineering*, vol. 11, no. 5s, pp. 631–635, Mar. 2023 Available: https://ijisae.org/index.php/IJISAE/article/view/7100

[32] Vladimir Batagelj and U. Brandes, “Efficient generation of large random networks,” *Physical Review E*, vol. 71, no. 3, pp. 036113–036113, Mar. 2005, doi: https://doi.org/10.1103/physreve.71.036113.

[33] “dwave-samplers — Python documentation,” *Dwavequantum.com*, 2025. https://docs.dwavequantum.com/en/latest/ocean/api_ref_samplers/index.html

[34] H. Wang, “Large-Scale Planar Graph Instances for Max-Cut Problem,” *Texas Digital Library (University of Texas)*, Aug. 2019, doi: https://doi.org/10.18738/t8/57q281.

[35] “Parameter Reference - Gurobi Optimizer Reference Manual,” *Gurobi.com*, 2026. https://docs.gurobi.com/projects/optimizer/en/current/reference/parameters.html#mipfocus.

[36] “CP-SAT Solver | OR-Tools,” *Google for Developers*. https://developers.google.com/optimization/cp/cp_solver.

[37] “Network Science by Albert-László Barabási,” *BarabásiLab*, 2026. https://networksciencebook.com/chapter/3#advanced-c.

[38] T.-J. . Chang, N. Meade, J. E. Beasley, and Y. M. Sharaiha, “Heuristics for cardinality constrained portfolio optimisation,” *Computers & Operations Research*, vol. 27, no. 13, pp. 1271–1302, Nov. 2000, doi: https://doi.org/10.1016/s0305-0548(99)00074-x.

[39] “Portfolio optimisation,” *Brunel.ac.uk*, 2026. https://people.brunel.ac.uk/~mastjjb/jeb/orlib/portinfo.html.